We focus on decision version of the problem that output either Yes/NO.

Ex: 1) Are given no. sorted?

2) Indep_Set: Given a graph and a set of vertices, check if they form indep Set.

3) Shortest path: Given a graph, s, t, threshold, check if ∃ a path from s to t of weight equal to threshold

If we can solve "decision version", then we can solve corresponding optimization version of the problem



Ex: Ask if ∃ a path of length 30 (Initial guess) from s to t — No
" 31 " " " — No
" 32 " " " — No
" 33 " " " — Yes

Shortest path length is 33

Ex Graph Coloring Problem: Input G, Min no. of colors required to color the nodes of vertices s.t no two adjacent nodes are of same color.] OPT

Decision Version: Given G and some +ve integer K, Can graph G be colored with K colors ?

Having oracle access to the soln of decision version of problem we can solve corresponding optimization version of problem. BINARY SEARCH

SAT Problem

n # of boolean variables x, y, z - - - , m literals

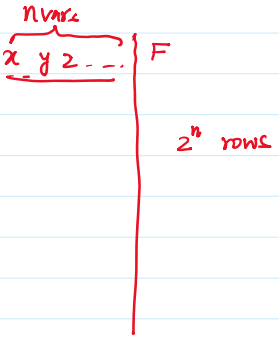
F := (x + y + z) (OR) AND (x + ȳ + z) AND (p + q + z) AND ...

literal — m literals

x = 0
y = 1
z = 1
.
.

Running

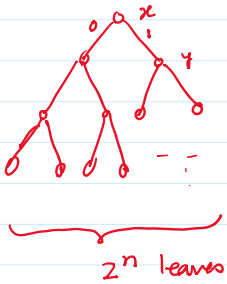
Question. what assignment of $x, y, z, \ldots$ will make the formula satisfiable.



Brute force Create a truth table over n variable (2^n possibilities). Check all the 2^n assignments one by one, and verify if they satisfy the formula.

Running time $3m \times 2^n$

Each path from root to leaf node is a row in the truth table



Suppose ∃ a machine that can evaluate all 2^n possible assignments in parallel. And if any one of them output Yes. The Final output is Yes.

All 2^n possible instance are running in parallel. And the running time for evaluating all 2^n instances is equal to running time of evaluating one instance.

(NP)

This is called non-determinism and algo is called non-deterministic algo

Running time of evaluating SAT = checking if one input instance make the formula satisfiable

$= 3m$ (poly in input size)

Non-deterministic poly time algo

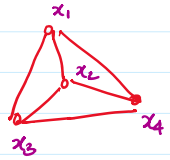
Equivalent def Guess a soln and verify if the soln is correct in poly time

Two step
① Given a guess of the soln
② Verify if the guess is correct?

" Equivalent to checking all guesses in parallel".

Clique Problem **Input** Graph, and a no k (not fixed its part of input)

Output Yes if ∃ a subgraph of size k which is a clique



Non-deterministic algo

Non-deterministic algo

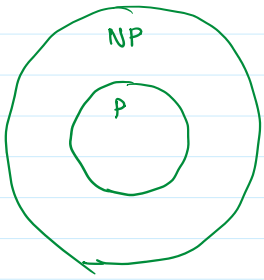


① Guess k nodes $O(k)$
② check if k nodes form a clique $O(k^2)$
] - poly (input size)

Binary Search, Sorting, cycle detecting, MST $\in P$

P: class of problem that can be solved by deterministic algo in poly time

NP: class of problems that can be solved by non-deterministic machine in poly time



3SAT, Clique $\in$ NP

NP-Complete: A problem is NP-Complete if it is in NP, and it is at least as hard as any other problem in NP.

> L is NP Complete $\Leftrightarrow$ $L \in NP$, $\forall A \in NP$, $A \leq_p L$

Reduction

$A \leq_p B$ — B is atleast as hard as A

A: known Problem
B: Unknown Problem
$\leq_p$: Running time of reduction

A is poly time reducible to B ie. $A \leq_p B$ if $\exists$ a poly time computable function f

s.t $x \in A \Leftrightarrow f(x) \in B$

x is instance of prob for which ans is Yes

The ans of problem A on X is Yes iff ans of prob B on f(x) is YES.

$B \leq_p A$ — A: known problem, B: Unknown problem — B is easier than A..

Theorem $NP \leq_p SAT \leq_p 3SAT$ (SAT: already known NP-complete prob)

Cook levin theorem

Proof SAT: any no. of variable in a literal

variables x, y, z, ...

$(x_1 \vee x_2 \vee x_3) \wedge (\quad)$

3SAT $\in$ NP — $x_1 \ldots x_n$ variable

(i) Nondeterministic machine guess a soln
≡ guessing one assignment of all the variable
$O(n)$

(ii) Verify the guess is correct
$O(m)$

(ii) Verify the guess is correct
O(m)
Overall O(n+m) □

Variables x, y, z, ...

(x) OR
$\wedge (y + \bar{x} + w + \bar{u})$
$\wedge (x + y + z)$
$\wedge (x + \bar{z})$
.
.

① $x \Leftrightarrow (x + a + b)$
$\wedge (x + a + \bar{b})$
$\wedge (x + \bar{a} + b)$
$\wedge (x + \bar{a} + \bar{b})$
a, b are auxillary variables

② $(x + \bar{z}) \Leftrightarrow (x + \bar{z} + c) \wedge (x + \bar{z} + \bar{c})$ c auxillary variable

③ $(y + \bar{x} + w + u) \Leftrightarrow (y + \bar{x} + L) \wedge (\bar{L} + w + u)$ L " "

④ $(x + \bar{y} + w + \bar{z} + u + v) \Leftrightarrow (x + \bar{y} + L_1) \wedge (\bar{L_1} + w + L_2) \wedge (\bar{L_2} + \bar{z} + L_3) \wedge (\bar{L_3} + u + L_4)$
$\wedge (\bar{L_4} + v + L_5)$
$L_1, L_2 .. L_5$ are auxillary variables

By ①, ②, .. ④ it is easy to see that if
SAT instance is satisfiable iff 3-SAT is satisfiable

Need three things :

① Given a SAT instance (already known NP-complete) problem construct 3-SAT instance.
also show that reduction time is poly

② If SAT instance is satisfiable, then corresponding 3 SAT instance is satisfiable

③ If 3-SAT instance is satisfiable, then SAT instance is satisfiable

Remains to show that reduction is poly time?
One variable - 4 clause
Two variable - 2 clause
4 " 2 "
n variable - O(n)

The reduction time remains poly.

Unknown Problem A
① developing efficient algo | ② If A is NP-complete?

Given an unknown decision problem A. Following are steps to show that A is NP complete

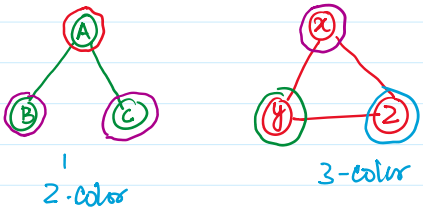
① $A \in NP$: Problem A can be solved using non-deterministic machine in poly time.

② $B \leq_p A$

↓

Known NP-complete problem

3-Colorability: Given a graph $G=(V,E)$ verify if the graph can be colored using 3-colors (No two adjacent nodes can have same color)



Theorem 3-Colorability $\leq_p$ SAT

↓

already known NP complete problem

Proof Given a graph need to construct a boolean formula s.t.

(1) if graph is 3-colorable, then formula is satisfiable

(2) if formula is satisfiable, then graph is 3-colorable

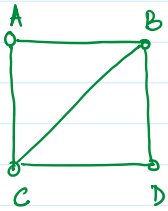
(3) Reduction time is poly.

"n nodes and e edges"

A_R := A colored with red

A_B := " " Blue

A_G := " " Green



$(A_R \bar{A}_B \bar{A}_G \vee A_G \bar{A}_R \bar{A}_B \vee A_B \bar{A}_R \bar{A}_G) \wedge$

Similar expression for all the nodes

↓ ensure that A can be colored using only one color R|G|B

Now need to incorporate the AB is an edge i.e. A & B can't be colored using same color.

If X, then Y $\quad X \rightarrow Y \equiv \bar{X} \vee Y$

$A_R \rightarrow \bar{B}_R$, $B_R \rightarrow \bar{A}_R$ Six clauses indicate one edge

$A_G \rightarrow \bar{B}_G$, $B_G \rightarrow \bar{A}_G$

$A_B \rightarrow \bar{B}_B$, $B_B \rightarrow \bar{A}_B$

$(\bar{A}_R \vee \bar{B}_R) \wedge (\bar{A}_G \vee \bar{B}_G) \wedge (\bar{A}_B \vee \bar{B}_B) \wedge (\bar{B}_R \vee \bar{A}_R)$

$A_G \to \bar{B}_G$, $B_G \to \bar{A}_G$

$A_B \to \bar{B}_B$, $B_B \to \bar{A}_B$

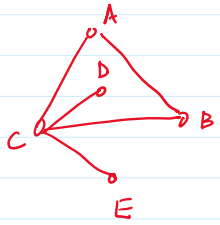
$(\bar{A}_R \vee \bar{B}_R) \wedge (\bar{A}_G \vee \bar{B}_G) \wedge (\bar{A}_B \vee \bar{B}_B) \wedge (\bar{B}_R \vee \bar{A}_R)$

$\wedge (\quad) \wedge (\quad)$

Running time of reduction $3n+6e = O(n+e)$

① & ② are H.W.

IndepSet: Given a graph $G=(V,E)$ indepset is a subset of vertices s.t no two vertices have an edge between them.



Indep set $\{A,D,E\}$, $\{B,D\}$, $\{B,D,E\}$

Theorem IndepSet is NP-complete.

Decision version: Does graph has IndepSet of size K?

① IndepSet $\in$ NP.

Proof: (i) Guess a soln - a subset of K vertices $O(K)$

(ii) Verify the soln - Check if those K vertices have an edge between them $O(K^2)$

} overall $O(K^2)$

Theorem 3SAT $\leq_p$ IndepSet

Known NP-complete prob.

Proof: Given a 3-SAT formula ϕ, construct a graph G_ϕ and a no. K st if ϕ is satisfiable then G_ϕ has an indep set of size K, and vice versa.

$\phi := (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_3 \vee x_4)$

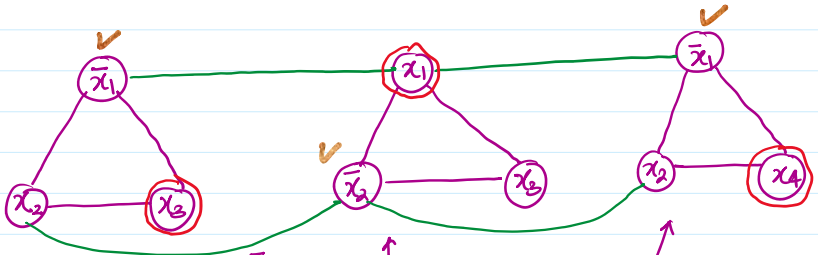
$x_3=1$, $x_1=1$, $x_4=1$, $x_2=1$, $\bar{x}_2=1$

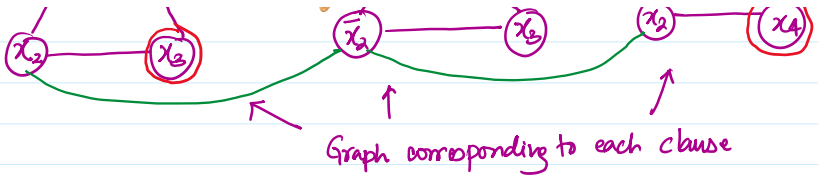
$x_1=1$ $\quad x_3=1$

$x_2=1$ $\quad x_4=1$

Pick a variable from each clause and find a truth assignment to make all of them true. This will fail if we pick two literals that are conflicting — x_i and $\bar{x}_i$

$G_\phi :=$





Graph corresponding to each clause

- One vertex for each variable in the clause
- Connect 3 variable in a clause by a triangle – indep set can take atmost one vertex from each triangle
- Connect two variable if they are complement of each other – x_i and $\bar{x}_i$
- Take K as no # of clauses

Claim: If ϕ is satisfiable, then G_ϕ has an indep set of size K (# of clauses in ϕ) and vice versa.

Proof. $\Rightarrow$ Pick one of the vertices from each clause (to the assignment of variable) or triangle

If ϕ is satisfiable then we can pick one vertex from each triangle

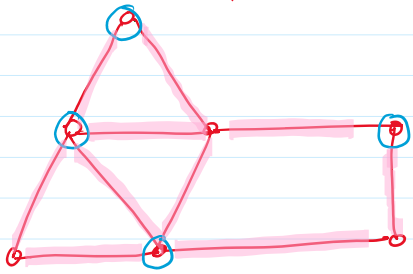
$\Leftarrow$ G_ϕ has an indep set S of size K

- S must contain one vertex from each triangle
- S will not contain vertices which have complement labels
- assign values to those vertices that make the clause corresponding to triangle satisfiable. And since we have one variable for each clause their assignment will make ↑ indep formula satisfiable.

(VC) Vertex Cover (nodes cover edges)
(NP-complete)

(Decision Version) Given a graph $G=(V,E)$ and a no. K, Can the graph be covered by K vertices

(Optimization) Given a graph $G=(V,E)$ finding min no of vertices that cover all the edges.

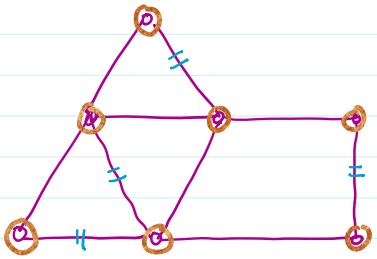


Edge Cover (Edge covers nodes)

(Decision Version) Given a graph $G=(V,E)$ and a no. K, Can all the nodes of G be covered by K edges

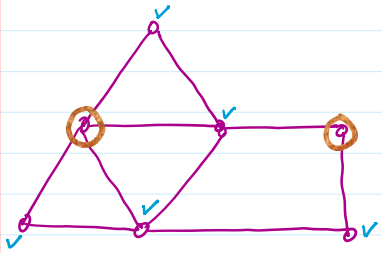
(Optimization) Given a graph $G=(V,E)$ finding min no of edges required to cover all the vertices

(Optimization) Given a graph $G=(V,E)$ finding min no of edges required to cover all the vertices



/* poly time algo using Max flow */

(DS) Dominating Set (Node cover nodes) (NP-complete)



Decision Version) Given a graph $G=(V,E)$ and a no. K, can all the nodes of G be covered by K nodes

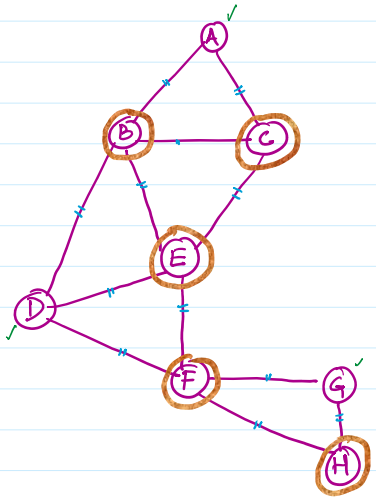
(Optimization) Given a graph $G=(V,E)$ finding min no of nodes required to cover all the vertices

Picking nodes to cover the graph (node | edges) is NP-complete
" edges " nodes is easy.

Theorem $VC \leq_p DS$.

Proof

VC



VC of size 5.
G K

G' f(k)

* If G has VC of size K then G' has DS of size f(k), and vice versa
* G' can be constructed in poly time

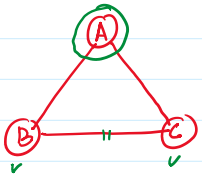
* If G has VC of size K then G' has DS of size f(K), and vice versa
* G' can be constructed in poly time

Attempt 1 Do nothing report G' = G and f(K) = K

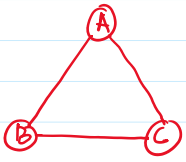
$\Rightarrow$ hold true

$\Leftarrow$ Not true

Vertex A is dom set but it doesn't cover edge BC

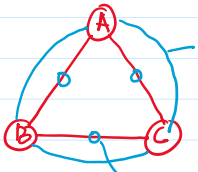


Reduction



G

$\Rightarrow$



extra edges

G'

extra vertex

Complete the proof H.W.